Detecting Forged Handwritten Signatures with a Reference-Based Convolutional Neural Network

UCS321 AI for Engineers, Report 1 | Akshara Thapan | Roll No. 7025000032 | Topic T-178

1. Introduction

Banks, courts and offices still accept handwritten signatures, and a human examiner checking them is slow and inconsistent. A model that flags likely forgeries could direct that attention where it is needed. My allocated topic, detecting forged handwritten signatures, is a binary image classification task: given a signature, decide whether it is genuine or a forgery. I made one requirement central from the start: the system must work on people it has never seen, because a bank cannot retrain a model for every new customer.

2. Dataset

I used the CEDAR signature database [1]: 55 signers, each with 24 genuine signatures and 24 skilled forgeries, scanned at 300 dpi. The notebook loads a public GitHub copy pinned to a single commit [2], because the official file is a RAR archive. The copy held 2,646 files, of which 6 were stray test files that were exact copies of other images, so I removed them, leaving 2,640. The features are the pixels and the target is genuine (0) or forged (1). Every signer contributes the same number of each class, so random guessing scores 50% and accuracy is a fair metric.

The most important decision in this project came from a routine check. Before training, I measured each image's background brightness. Figure 1 shows that genuine images have a median grey level near 239 and forged images near 254, with no overlap. This has nothing to do with handwriting: the two groups appear to have been scanned or processed differently. A one-line rule using brightness alone scored 100.0% on my test set, without looking at a single stroke. The real difficulty is closer to the right panel, where ink coverage barely separates the classes.

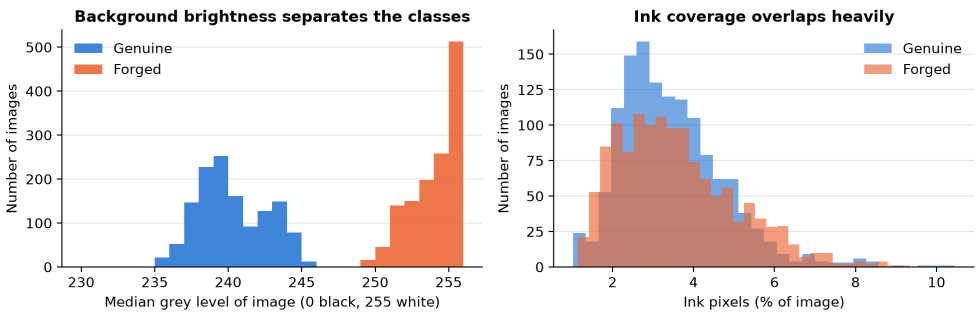


Figure 1: Background grey level (left) separates the classes completely across 2,640 images, while ink coverage (right) overlaps heavily.

3. Methodology

Pre-processing was therefore designed around removing this shortcut. I applied Otsu's threshold [4] to each image to keep only the ink as a black-and-white mask, cropped to the signature and resized it to 96 × 192 pixels with padding so that its shape was kept. Every background is now identical. The grey version was kept only as a control. The IQR rule flagged 102 images on aspect ratio and 61 on ink coverage. I kept them, since long or sparse signatures are real, and the padding handles their shape.

I split the data by signer rather than by image: 39 signers for training, 8 for validation and 8 for testing, so nobody appears in two sets. I chose this because a random image split would test the model on people it had already met. To measure the effect, I also retrained one model on a random split.

The topic names a CNN, so I built three from the same four convolutional blocks. The single-image CNN sees only the signature in question. The Reference CNN, my main model, receives that signature together with a genuine signature of the same person as a two-channel image, and its prediction is averaged over five enrolled references. I chose this design because real verification compares against a specimen on file, and the idea follows the writer-independent approach of SigNet [3]. A twin (Siamese) network was also tried, but it was slower and sat near 60% after five epochs in an early trial, so I stopped it. The third CNN used raw images, only to measure the artefact. Training used Adam with a cosine learning-rate schedule, dropout and small random rotations, scalings and shifts, because nobody signs identically twice. The best epoch was chosen on validation accuracy.

For comparison I used Logistic Regression, an RBF SVM and a Random Forest on shrunken images, plus a Reference SVM that receives the same reference information as the Reference CNN. These were implemented in PyTorch and scikit-learn [5].

4. Results

Table 1 shows that the Reference CNN was the best valid model, with 78.8% accuracy (271 of 344) and an ROC AUC of 0.861. This is slightly under the expected 80%. The single-image CNN reached 70.1%, so seeing a reference added 8.7 points. The classical models stayed between 47.4% and 58.1%, and the Reference SVM reached only 55.5% despite the same input, which suggests that comparing raw pixels is not enough and that the convolutional filters are doing the useful work. The raw-image control scored 100.0%. I report it but treat it as invalid, since Figure 1 explains how it was achieved.

Table 1: Test results on the same 344 signatures from eight unseen signers, with forged as the positive class. FAR is the share of forgeries accepted and FRR the share of genuine signatures rejected. *Invalid: it reads the scanner artefact.

Model	Acc.	Prec.	Rec.	F1	AUC	FAR	FRR
Reference CNN	**0.788**	**0.805**	**0.818**	**0.811**	**0.861**	**0.182**	**0.250**
Single-image CNN	0.701	0.694	0.828	0.755	0.792	0.172	0.461
Random Forest	0.581	0.638	0.578	0.607	0.564	0.422	0.414
Reference SVM	0.555	0.577	0.760	0.656	0.606	0.240	0.704
SVM (RBF)	0.546	0.596	0.583	0.590	0.574	0.417	0.500
Logistic Regression	0.474	0.531	0.484	0.507	0.479	0.516	0.539
Raw control*	1.000	1.000	1.000	1.000	1.000	0.000	0.000

The confusion matrix in Figure 2 shows that errors are fairly balanced: 35 forgeries were accepted (FAR 18.2%) and 38 genuine signatures rejected (FRR 25.0%). The AUC of 0.861 means the probabilities rank signatures better than the 0.50 cut-off suggests, so a bank could lower the cut-off to catch more forgeries at the cost of rejecting more customers. Results differ widely by person, from 60.5% to 100.0% across the eight test signers, so some signatures are clearly easier to verify than others.

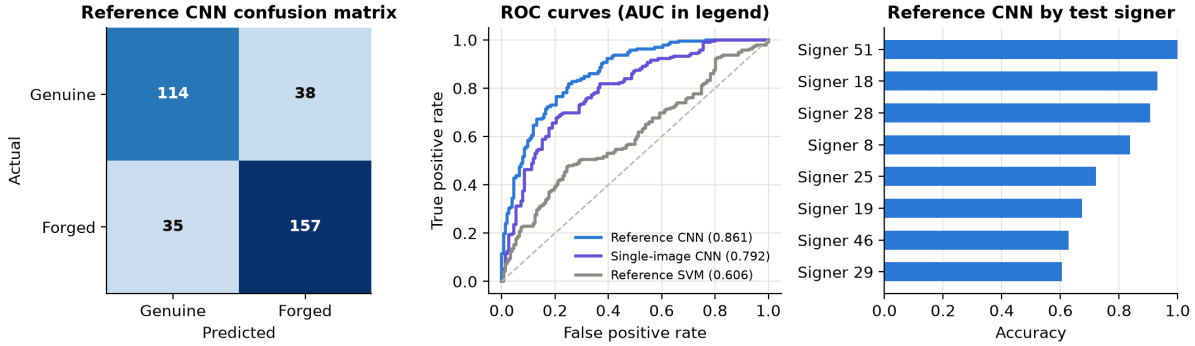


Figure 2: Reference CNN on test signers: confusion matrix (left), ROC curves with AUC in legend (centre) and accuracy by signer (right).

Two further checks test these results. Repeating the Reference CNN on three different signer splits gave accuracies of 78.8%, 75.0%, 73.3% (mean 75.7%, standard deviation 2.8 points), so the choice of signers matters. For the leakage check, the single-image CNN scored 76.7% on a random image split against 70.1% on the signer split, a difference of 6.7 points.

5. Discussion and Conclusion

The best model is the Reference CNN, and my main finding is about method rather than architecture. A careless pipeline would have reported 100.0%, and that number would have been meaningless. After the shortcut was removed, the honest figure is 78.8% on strangers. It falls just short of 80%, and I believe the reasons are real: only 39 training signers, skilled forgeries written to imitate the target, and the removal of the easiest cue. The noisy validation curves also suggest that the model had not fully settled. I did not keep adjusting the model against the test set to pass 80%, because that would make the test score unreliable.

There are clear limitations. Eight test signers is a small sample, and binarising also discards pen-pressure information that may be useful. The artefact suggests each class came from its own scanning setup, so these results may not transfer to other scanners. With more time, I would try a pretrained network, which I did not test here, train on a larger multi-writer dataset, and choose the cut-off from the real cost of a missed forgery. The wider lesson for me is that a perfect score should be investigated, not celebrated.

References

[1] M. K. Kalera, S. Srihari and A. Xu, “Offline signature verification and identification using distance statistics,” Int. J. Pattern Recognition and Artificial Intelligence, vol. 18, no. 7, pp. 1339–1360, 2004.

[2] CEDAR signature images. Official: https://cedar.buffalo.edu/NIJ/data/signatures.rar. GitHub copy used: https://github.com/Snehal-Aparadh/SignatureForgeryDetection (commit 9e4157b).

[3] S. Dey et al., “SigNet: Convolutional Siamese network for writer independent offline signature verification,” arXiv:1707.02131, 2017.

[4] N. Otsu, “A threshold selection method from gray-level histograms,” IEEE Trans. Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62–66, 1979.

[5] PyTorch, https://pytorch.org; F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

AI tool use: Claude (Anthropic) was used to help design the experiments, write the notebook code and draft this report.